| | |
|---|---|
| **Project Title** | **Reel Insights: An End-to-End Exploratory Data Analysis of the Global Movie Industry (TMDB)**<br><br>*SQL • Python (Pandas) • Visualization • Streamlit* |
| **Skills Takeaway** | • Working with a multi-file relational CSV dataset (6 linked tables)<br>• Python (Pandas, NumPy) for data wrangling and feature engineering<br>• Data Cleaning & Relational Schema Design<br>• SQL & Relational Databases (MySQL / PostgreSQL / SQLite)<br>• Database connectivity using PyMySQL, mysql-connector-python, psycopg2, sqlite3<br>• Exploratory Data Analysis (EDA) — univariate, bivariate , multivariate<br>• Data Visualization with Matplotlib, Seaborn and Plotly<br>• Interactive multi-page dashboarding with Streamlit<br>• Insight generation, business storytelling & recommendation writing |
| **Domain** | **Media & Entertainment Analytics (Box-Office / Content Analytics)** |

## Problem Statement

As a Data Analyst, you have been provided with six relational CSV files covering ~2,500 movies from The Movie Database (TMDB) — movies, genres, cast, crew and keywords. Your task is to load and clean this dataset, structure it into a proper relational SQL database, and carry out a full end-to-end analysis pipeline — SQL querying, Pandas EDA, visualization, and a final multi-page dashboard built in Streamlit.

## Real World Use Cases

Understanding why this kind of analysis matters in the industry helps connect the technical work to real business impact:

- Box-Office Forecasting Support — Studios use budget/genre/cast history to gauge a new film's commercial risk before greenlighting.

- Content Acquisition (Streaming Platforms) — Platforms like Netflix/Prime analyze genre performance and rating trends to decide what content to license or produce.
- Talent & Casting Analytics — Production companies evaluate an actor's or director's historical box-office/rating track record before casting decisions.
- Recommendation Engines — Genre, keyword and cast metadata (exactly what this project collects) power the similarity logic behind "movies you might like".
- Marketing Spend Optimization — Marketing teams correlate genre and rating patterns with audience reception to plan release-date and promotional strategy.
- Film Critique & Journalism — Entertainment journalists and review aggregators use rating/vote-count trends to write industry trend pieces.

## Dataset Collection

You have been provided with six CSV files, already structured into a relational schema. Place all six in a data/ folder before starting.

| File | Contents | Approx. Rows |
|---|---|---|
| **Dataset Link** | TMDB_data | - |
| **movies.csv** | One row per movie — title, release date, budget, revenue, runtime, popularity, rating, vote count, status and overview. | ≈ 2,500 rows |
| **genres.csv** | Reference list of all movie genres (e.g., Action, Drama, Comedy) and their IDs. | ≈ 19 rows |
| **movie_genres.csv** | Bridge table linking each movie to its genre(s) — a movie can have more than one genre. | ≈ 5,000–6,000 rows |
| **cast.csv** | Top-billed actors per movie, with character name and billing order. | ≈ 20,000+ rows |
| **crew.csv** | Director and writer credits per movie. | ≈ 4,000–5,000 rows |
| **movie_keywords.csv** | Thematic tags/keywords associated with each movie. | ≈ 8,000–10,000 rows |

### Steps to follow:

- Load all six CSVs into Pandas DataFrames and inspect each one's shape, column data types, and null counts.
- Clean each table — fix data types (e.g., parse release_date as a date, ensure IDs are integers), remove exact duplicate rows, and remove orphaned rows (e.g., a cast row referencing a movie_id that no longer exists in movies.csv after cleaning).
- Do not silently 'fix' budget/revenue values of 0 — TMDB uses 0 to mean the figure is unknown, not that the movie was free. How to handle this is something you'll decide as part of the Pandas analysis section.
- Once cleaned, load all six tables into a SQL database (SQLite /MySQL/PostgreSQL) using sqlite3, pymysql, mysql-connector-python or psycopg2, sqlalchemy.
- Run one simple test query (e.g., a JOIN across movies, movie_genres and genres) to confirm the database loaded correctly before moving on to the full SQL analysis section.

## Table Schema Reference

The six tables below form the relational structure of the dataset. Use this as your reference when writing SQL joins and Pandas merges throughout the project.

GUVI | HCL
Skill Up. Level Up

### movies

Provided file: movies.csv — target ≈ 2,500–3,000 rows

| Column | Data Type | Description |
|---|---|---|
| movie_id | INT (PK) | Unique TMDB movie identifier |
| title | VARCHAR(255) | Movie title |
| original_language | VARCHAR(10) | ISO language code, e.g. 'en' |
| release_date | DATE | Theatrical release date |
| runtime | INT | Runtime in minutes |
| budget | BIGINT | Production budget in USD |
| revenue | BIGINT | Box-office revenue in USD |
| popularity | FLOAT | TMDB popularity score |

| | | |
|---|---|---|
| vote_average | FLOAT | Average user rating (0–10) |
| vote_count | INT | Number of user votes |
| status | VARCHAR(50) | e.g. Released, Post Production |
| overview | TEXT | Short plot summary |

### genres

Provided file: genres.csv — static reference table, ≈ 19 rows

| Column | Data Type | Description |
|---|---|---|
| genre_id | INT (PK) | Unique genre identifier |
| genre_name | VARCHAR(50) | e.g. Action, Drama, Comedy |

GUVI | HCL

### movie_genres

Provided file: movie_genres.csv — ≈ 5,000–6,000 rows

| Column | Data Type | Description |
|---|---|---|
| movie_id | INT (FK → movies) | References movies.movie_id |
| genre_id | INT (FK → genres) | References genres.genre_id |

### cast

Provided file: cast.csv (top 10 billed actors per movie) — ≈ 20,000–25,000 rows

| Column | Data Type | Description |
|---|---|---|
| cast_row_id | INT (PK, auto-increment) | Surrogate row key |
| movie_id | INT (FK → movies) | References movies.movie_id |
| person_id | INT | TMDB person identifier |
| actor_name | VARCHAR(255) | Actor's name |

| | | |
|---|---|---|
| character_name | VARCHAR(255) | Character played |
| cast_order | INT | Billing order (0 = top-billed) |

### crew

Provided file: crew.csv (Director + Writer credits per movie) — ≈ 4,000–5,000 rows

| Column | Data Type | Description |
|---|---|---|
| crew_row_id | INT (PK, auto-increment) | Surrogate row key |
| movie_id | INT (FK → movies) | References movies.movie_id |
| person_id | INT | TMDB person identifier |
| person_name | VARCHAR(255) | Crew member's name |
| job | VARCHAR(100) | e.g. Director, Writer, Producer |
| department | VARCHAR(100) | e.g. Directing, Writing |

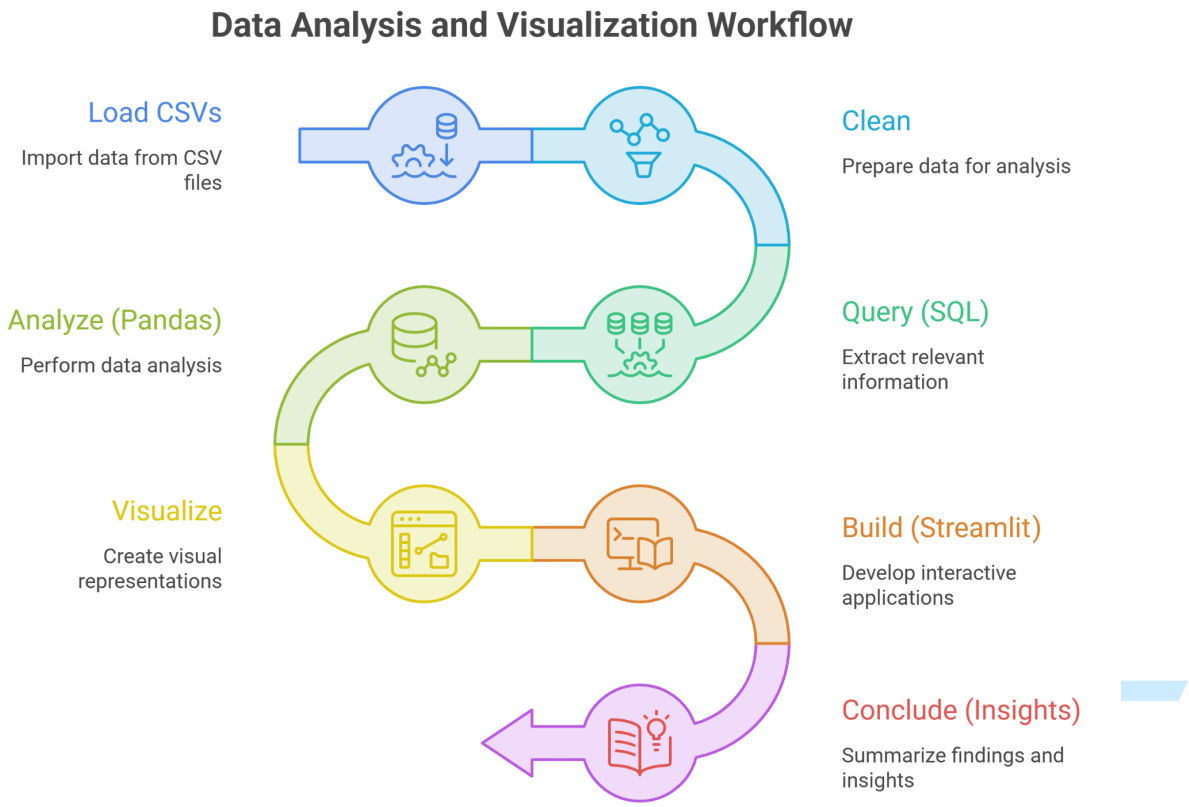
GUVI HCL
Skill Up. Level Up

### movie_keywords

Provided file: movie_keywords.csv — ≈ 8,000–10,000 rows

| Column | Data Type | Description |
|---|---|---|
| movie_id | INT (FK → movies) | References movies.movie_id |
| keyword_id | INT | TMDB keyword identifier |
| keyword_name | VARCHAR(100) | e.g. 'time travel', 'heist' |

## Project Flow




## Notebook & File Structure

Submit a well-organized project folder. Every notebook must use markdown headers to separate sections, contain clear inline comments, and end each analysis with a short written insight/recommendation — not just code output.

- 01_data_cleaning.ipynb — Load the six provided CSVs, handle nulls/placeholder values, remove duplicates and orphaned rows, enforce data types, and load the cleaned tables into your SQL database.
- 02_sql_analysis.ipynb — Connect to your chosen database (sqlite3 / pymysql / mysql-connector-python / psycopg2), load the cleaned tables, and answer all 10 SQL questions
- 03_pandas_analysis.ipynb — Answer all 10 Pandas analysis questions, each followed by a short insight.
- 04_visualizations.ipynb — Univariate, bivariate and multivariate visualizations using Matplotlib/Seaborn/Plotly, each with a 2–4 sentence insight.

- app.py — final deliverable: a multi-page Streamlit app that reads from your SQL database, with Filters, Dashboard and Query Explorer pages

## SQL Analysis — 10 Questions

Use sqlite3, pymysql, mysql-connector-python or psycopg2 (pick one database) to connect from Python and execute these queries. Questions progress from simple filtering/joins to multi-table joins, GROUP BY/HAVING, and subqueries.

| # | Question | Concept Focus |
|---|---|---|
| 1 | List all movies in the Action genre released after 2015, showing title and release date. | JOIN (movies ⋈ movie_genres ⋈ genres) |
| 2 | Find the top 10 highest-grossing movies along with the genre(s) each belongs to. | JOIN + ORDER BY + LIMIT |
| 3 | Calculate the average budget and average revenue for each genre, sorted by average revenue descending. | JOIN + GROUP BY + AVG() |
| 4 | List the top 10 actors who have appeared in the most movies in the dataset.(cast) | GROUP BY + COUNT() |
| 5 | Find all directors who have directed more than 3 movies, along with their movie count. | GROUP BY + HAVING |
| 6 | List the top 10 most frequently used keywords across all movies. | GROUP BY + COUNT() |
| 7 | Find movies where the budget was above the overall average budget but the revenue was below the overall average revenue. | Subquery comparison against AVG() |
| 8 | List all actors who have appeared in a movie directed by a specific director (e.g., Christopher Nolan). | JOIN (cast ⋈ crew ) |
| 9 | Find the genre with the highest average rating, considering only genres with at least 20 movies. | JOIN + GROUP BY + HAVING + AVG() |
| 10 | List the top 10 movies with the highest number of keywords tagged to them. | JOIN + GROUP BY |

GUVI | HCL Skill Up. Level Up

## Pandas Analysis — 10 Questions

| # | Question |
|---|---|
| 1 | List the total number of unique movies, genres, cast members and crew members collected across all tables. |
| 2 | List all movies with missing or zero budget/revenue values, and decide how these should be treated for the rest of the analysis. |
| 3 | List the profit (revenue − budget) and return on investment for every movie that has valid budget and revenue figures. |
| 4 | List the top 15 movies by return on investment, considering only movies with a budget of at least $1 million. |
| 5 | List the number of movies released per year, and identify which year had the highest movie output in the dataset. |
| 6 | List all movies whose runtime is a statistical outlier compared to the rest of the dataset. |
| 7 | List each movie's primary genre (the first genre listed) and compare average popularity across primary genres. |
| 8 | List the top 10 most prolific actors by number of movies, along with their average movie rating across those movies. |
| 9 | List any movie titles that appear more than once in the dataset, along with their release years, to check for duplicates or remakes. |
| 10 | List movies whose popularity score and vote count don't follow the general pattern between the two — i.e., unusually high popularity with a low vote count, or vice versa. |

## Visualization Tasks

---

Use Matplotlib, Seaborn and/or Plotly (any/all). Below are idea prompts only — explore broadly across univariate, bivariate and multivariate analysis, choosing appropriate chart types. For every chart, write insight — not axis descriptions.(8-12)

### Univariate

- Distribution of movie budgets (histogram/KDE).
- Distribution of runtimes (box plot / violin plot).
- Distribution of vote_average scores (histogram).
- Count of movies per genre (bar/count plot).

### Bivariate

- Budget vs revenue (scatter plot, with trend line).
- Popularity vs vote count (scatter plot).
- Average rating by genre (bar chart).
- Number of movies released per year (bar/column chart).

### Multivariate

- Correlation heatmap of budget, revenue, runtime, popularity, vote_average and vote_count (heatmap).
- Pairplot of budget, revenue, popularity and vote_average, colored by primary genre (pairplot).
- Budget vs revenue colored by primary genre, with point size showing vote_count (scatter plot with hue and size).
- Distribution of vote_average for each genre in separate panels (FacetGrid with histograms).
- Average revenue by genre and release decade (heatmap of a pivot table).

GUVI | HCL
Skill Up. Level Up

## Streamlit Application — Page Structure

---

Build a multi-page Streamlit app with three pages, all working off the same underlying database.

### Page 1 — Filters

- A dedicated Filters page (or sidebar, shared across all pages) with: genre selectbox/multiselect, release-year range slider, minimum-rating slider, and language selectbox.
- Selected filter values should be stored in st.session_state so they persist and apply consistently across the Dashboard and Query Explorer pages.
- Show a live count of how many movies match the current filter selection.

### Page 2 — Dashboard

- KPI cards (st.metric) for: total movies, total genres represented, total actors, and average rating — all reflecting the current filter selection.
- Top 10 movies by revenue (bar chart) for the filtered selection.
- Genre distribution (pie/donut or bar chart) for the filtered selection.
- Budget vs revenue scatter plot for the filtered selection.
- A short auto-generated text insight summarizing the filtered view (e.g., "highest grossing genre in this selection is...").

**Page 3 — Query Explorer :**

- Dropdown of saved queries
- Show results as **st.dataframe** and a **chart**
- Text insight box below

## Submission Guidelines

- Submit app.py (the Streamlit app)
- Submit a single project folder containing all notebooks and app.py, named clearly as listed in the Notebook & File Structure section.
- Keep the data/ folder (all six CSVs plus your SQL database file) alongside your notebooks so the project runs end-to-end for anyone reviewing it.
- Every notebook must have markdown section headers, inline code comments, and a written insight/recommendation block after each analysis.
- Include a short README summarizing your top 5 overall insights and recommendations across the whole project.

## Project Deliverables

| Deliverables | Streamlit App |
|---|---|
| Notebooks (01–04) | Same — complete all 4 |

| | |
|---|---|
| Final Tool | Streamlit app (`app.py`) |
| How Data Gets In(for queries) | App reads from your SQL database using Python |
| What You Build | 3 pages: Filters, Dashboard, Query Explorer |
| SQL Question Coverage | All 10 questions, shown as plain tables and charts in Query Explorer |

---

## Technical Tags

**Python, Pandas, NumPy, Matplotlib, Seaborn, Plotly, SQL, PostgreSQL, MySQL, SQLite, Data Cleaning, EDA, Streamlit, Data Visualization, Feature Engineering, Relational Database Design, Media & Entertainment Analytics, Box-Office Analytics**

---

## ⏳ Timeline

The project should be completed and submitted within 10 - 14 days from the date it is assigned.

| Created By | Verified By | Approved By |
|---|---|---|
| Nilofer Mubeen | Selvamani A | Nehlath Harmain |